SRM UNIVERSITY

DELHI-NCR, SONEPAT • DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Academic Year 2026–27

ENTERPRISE CLOUD SYSTEMS ENGINEERING REPORT

CloudOps Lab

CloudOps Lab

Automated AWS Infrastructure Analysis & Network Self-Healing Platform

Automated AWS Infrastructure Analysis & Network Self-Healing Platform

An evidence-based cloud networking diagnostic and automated remediation platform. Models Virtual Private Cloud (VPC) SDN primitives, audits Layer 1–4 configuration invariants, isolates deterministic root causes, synthesizes declarative Terraform remediation, and empirically certifies zero-regression

Comprehensive Technical Report & Enterprise System Architecture Specification

Author Identification:
Ashutosh Chaudhary (11723210015)
Diksh Grover (11723210013)
Divesh Sharma (11723210014)

Document Identification: CLOUDOPS-SPEC-TR-2026-V1

Project Name: CloudOps Lab (AWS Infrastructure Analysis & Network Troubleshooting Platform)

Classification: Enterprise Cloud Engineering & SRE Platform Specification

Version: 1.0.0 (Production Master)

Primary Repository: `https://github.com/a7hu-15/cloud_ops.git`

Live Production Host: `http://44.199.232.34/` (Amazon Web Services, Region `us-east-1`)

AWS Resource Group: `cloudops-lab-resources` (AWS Account ID: `757952140199`)

Date of Publication: November 2026

DEPLOYMENT SPECIFICATIONS:

Live Cloud Endpoint: http://44.199.232.34/

AWS Host Region: us-east-1 (US East)

AWS Resource Group: cloudops-lab-resources

Source Repository: https://github.com/a7hu-15/cloud_ops.git

Document ID: CLOUDOPS-SPEC-TR-2026-V1

Table of Contents

1. Executive Summary & Abstract
2. Enterprise Problem Statement & Operational Challenges
3. Platform Use Cases & Production Value Proposition
4. Platform Scope Boundaries & Operational Constraints
5. Complete Technology Stack & Architectural Principles
6. System Architecture & Multi-Tier Module Decomposition
7. Pre-Deployment Network Planning & CIDR Mathematics Engine
8. The 10-Point Automated Diagnostic Audit Catalogue
9. Deterministic Root-Cause Analysis (RCA) Engine
10. Declarative Infrastructure as Code (Terraform) Remediation Engine
11. Human-in-the-Loop Safety Guardrails & Access Governance
12. The End-to-End 7-Step Self-Healing Recovery Workflow
13. Hop-by-Hop Packet Path Tracer & Cloud Resource Inventory
14. Database Schema & Entity-Relationship Modeling (ER Diagram)

15. Authentication, Cryptography & Session Security Architecture
16. Production Containerization & Multi-Stage Docker Architecture
17. Automated Testing Suite & Continuous Integration Pipeline
18. Live Cloud Infrastructure & AWS Resource Group Organization
19. Audit Reporting, Compliance & Regulatory Verification
20. Operational Runbook & Administrator Guide
21. References, Industry Standards & Citations

1. Executive Summary & Abstract

Modern enterprise cloud deployments hosted on Amazon Web Services (AWS) depend upon deeply interconnected, software-defined network (SDN) primitives. Virtual Private Clouds (VPCs), IPv4 Classless Inter-Domain Routing (CIDR) blocks, public and private subnet tiers, route tables, perimeter Internet Gateways, Elastic Network Interfaces (ENIs), and stateful Security Groups form a continuous chain of dependencies. When a service outage occurs, over 70% of network failures stem from subtle human configuration errors: an unassigned default route, an overlapping subnet allocation, an omitted security group authorization, or a detached perimeter gateway.

To external clients, these diverse misconfigurations manifest identically: TCP connection timeouts, packet drops, or unreachable web endpoints. Traditional cloud monitoring systems (such as generic ping probes or synthetic HTTP checks) merely record the symptom—they fail to identify the underlying architectural fault or provide a safe path to recovery. Operations teams are consequently forced into hours of manual console navigation, cross-referencing disparate AWS screens, risking configuration drift, and executing unreviewed, unversioned fixes.

CloudOps Lab is an enterprise-grade automated AWS infrastructure analysis, diagnostic audit, deterministic root-cause isolation, and self-healing recovery platform. When an organization connects its AWS environment or network plan to CloudOps Lab: 1. The platform automatically models and visualizes the complete VPC architecture, subnet tiers, routing targets, and firewall policies. 2. It executes an automated **10-point deterministic diagnostic suite** covering Layer 1 through Layer 4 cloud networking invariants in milliseconds. 3. It utilizes a **multi-factor deterministic Root-Cause Analysis (RCA) engine** that systematically tests and eliminates healthy network layers, isolating the exact failing component with mathematical certainty and up to 100% confidence. 4. It synthesizes declarative **HashiCorp Terraform Infrastructure as Code (IaC)** patches to remediate the identified fault. 5. It enforces strict enterprise **Human-in-the-Loop safety guardrails**, requiring explicit operator review and token-based confirmation (`CONFIRM_REMEDIATION`) before changes are committed. 6. It executes automated **post-remediation recovery verification**, proving that

the infrastructure has returned to a 100% passing state with zero configuration regressions. 7. It issues tamper-proof, immutable **compliance audit reports** in GitHub Flavored Markdown and JSON formats for post-incident review and regulatory compliance.

The platform is deployed live on Amazon Web Services in Region `us-east-1` at `http://44.199.232.34/`, fully containerized via multi-stage Docker builds, verified by 55 automated unit and integration tests, and maintained via continuous integration workflows on GitHub (`https://github.com/a7hu-15/cloud_ops.git`).

---

2. Enterprise Problem Statement & Operational Challenges

2.1 The Architectural Complexity of AWS Software-Defined Networking

In an on-premises physical data center, networking components (switches, routers, physical firewalls, cables) are distinct, physical entities. In AWS, these physical layers are abstracted into software-defined constructs managed through APIs: * **Amazon VPC:** A logically isolated virtual network defined by an IPv4 CIDR block. * **Subnets:** Segments of a VPC's IP address range assigned to specific Availability Zones. * **Internet Gateways (IGWs):** Horizontally scaled, redundant VPC perimeter components providing bidirectional network address translation (NAT). * **Route Tables:** Dynamic routing rule tables directing packets to local destinations, internet gateways, virtual private gateways, or VPC peering endpoints. * **Security Groups:** Stateful virtual firewalls operating at the Elastic Network Interface (ENI) level, controlling port- and protocol-level ingress and egress traffic.

Because these software-defined resources are tightly coupled, an end-to-end network transaction succeeds only if **every single component in the path is correctly configured**:

$$\text{Path Success} = \text{CIDR}_{\text{valid}} \land \text{Subnet}_{\text{contained}} \land \text{IGW}_{\text{attached}} \land \text{Route}_{\text{default}} \land \text{SG}_{\text{authorized}}$$

If any single term in this Boolean conjunction evaluates to `False`, the entire communication path fails.

2.2 Critical Failure Modes in Cloud Operations

In enterprise production operations, troubleshooting these network failures presents significant operational challenges:

1. **The "Silent Drop" Symptom Equivalence:** When an external client attempts to connect to an EC2 instance on port 443 (HTTPS) and experiences a timeout, the root cause could be:

- The Security Group lacks an inbound authorization rule for TCP 443 (Layer 4 failure).
- The Route Table associated with the public subnet lacks a `0.0.0.0/0` next-hop to the Internet Gateway (Layer 3 failure).
- The Internet Gateway exists in the AWS region but is in a `detached` state from the VPC (Perimeter failure).
- The subnet CIDR is overlapping with an existing network, causing routing conflicts (Layer 1/2 failure). To traditional APM (Application Performance Monitoring) tools, all four scenarios generate the exact same error: `ETIMEDOUT`.

2. **High Mean Time to Detect (MTTD) and Resolve (MTTR):** Site Reliability Engineers (SREs) typically spend 45 to 120 minutes during a major incident clicking through disparate AWS Management Console menus (VPC Dashboard, Route Tables, Subnet associations, EC2 Security Groups, Network ACLs) to identify which component is blocking traffic.

3. **Configuration Drift and Production Risk:** Under the pressure of production outages, engineers frequently execute ad-hoc, unreviewed manual modifications directly in the AWS web console. An engineer attempting to fix HTTPS connectivity might inadvertently add an unrestricted rule (`0.0.0.0/0` on port 22), creating an immediate security vulnerability, or modify an active route table and disconnect adjacent production databases.

4. **Lack of Empirical Recovery Proof:** In standard operations, once a fix is applied, verification is ad-hoc (e.g., an engineer refreshing a browser tab). There is rarely an automated mechanism that re-audits all dependent network rules to certify that the primary failure has been cleared and that no secondary regressions were introduced.

---

3. Platform Use Cases & Production Value Proposition

CloudOps Lab addresses these enterprise challenges directly. The platform serves three primary organizational stakeholders: Site Reliability Engineers (SREs), Cloud Network Architects, and Cloud Security/Compliance Auditors.

3.1 Primary Enterprise Use Cases

Use Case A: Ingestion and Automated Audit of Live AWS Environments

- **Workflow:** An organization connects its AWS credentials (or supplies its infrastructure configuration model) to CloudOps Lab.
- **Platform Execution:** The ingestion engine queries AWS EC2 and VPC APIs, reconstructing the VPC boundaries, subnet tiers, routing paths, and security group rule sets.
- **Outcome:** The platform runs the 10-point diagnostic audit suite in parallel, producing an immediate status matrix highlighting operational health, identifying misconfigurations, and categorizing findings by severity (`CRITICAL`, `HIGH`, `MEDIUM`).

Use Case B: Pre-Deployment IPv4 CIDR Validation & Collision Prevention

- **Workflow:** Network architects design VPC address spaces prior to committing Terraform modules to production repositories.
- **Platform Execution:** The platform evaluates proposed CIDR allocations against RFC 1918 standards, enforces the AWS 5-reserved-IP rule, verifies strict subnet containment, and runs an $O(N^2)$ pairwise subnet intersection algorithm.
- **Outcome:** CIDR collisions and boundary violations are caught and rejected prior to cloud provisioning, eliminating failed Terraform applies and non-routable deployments.

Use Case C: Automated Incident Response & Root-Cause Pinpointing

- **Workflow:** A network connectivity outage occurs affecting public web workloads.

- **Platform Execution:** The diagnostic engine scans the environment and feeds the findings into the deterministic RCA engine. The RCA engine systematically evaluates layer dependencies (ruling out Layer 1 CIDR, Layer 2 perimeter, and Layer 3 routing) to isolate the exact failing firewall rule with 100% mathematical confidence.
- **Outcome:** SREs receive an instant, evidence-backed diagnosis with the exact resource ID, eliminating guesswork and slashing MTTD to under two seconds.

Use Case D: Controlled Self-Healing via Sandboxed Terraform Remediation

- **Workflow:** Following root-cause determination, the platform synthesizes declarative Terraform HCL code representing the precise patch required.
- **Platform Execution:** The code diff is displayed in the console. The platform enforces an explicit **Human-in-the-Loop guardrail**: the operator must inspect the diff and enter the safety confirmation token `CONFIRM_REMEDIATION`.
- **Outcome:** The patch is applied via sandboxed Terraform execution (`shell=False`), maintaining complete alignment with Infrastructure as Code practices and preventing unversioned console drift.

Use Case E: Empirical Post-Fix Verification & Compliance Archiving

- **Workflow:** After remediation is applied, the platform executes a comprehensive re-audit.
- **Platform Execution:** All 10 diagnostic rules are evaluated a second time against the updated state.
- **Outcome:** The platform certifies that the prior failing check is cleared (`Resolved: SG-001`), verifies that zero secondary failures remain (`Remaining: 0`), and commits an immutable, timestamped audit report to the relational database for compliance and SLA auditing.

---

4. Platform Scope Boundaries & Operational Constraints

To maintain architectural focus, predictable resource consumption, and deterministic reliability, CloudOps Lab defines explicit operational boundaries:

4.1 In-Scope Capabilities

- IPv4 Classless Inter-Domain Routing (CIDR) validation ($/16$ to $/28$ prefix masks).
- Automated enforcement of RFC 1918 private address ranges (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`).
- Calculation of the AWS 5-reserved-IP rule across all defined subnet bitmasks.
- Modeling of AWS VPCs, Subnets (Public and Private tiers), Route Tables, Internet Gateways, and Security Groups.
- Stateful Layer 4 Security Group rule parsing (inbound/outbound TCP/UDP/ICMP authorization).
- Automated 10-point diagnostic audit catalogue execution.
- Multi-factor deterministic root-cause analysis with confidence scoring.
- Sandboxed HashiCorp Terraform CLI execution (`validate`, `plan`, `apply`).
- Explicit Human-in-the-Loop safety confirmation guardrails (`CONFIRM_REMEDIATION`).
- Automated post-remediation regression verification.
- Relational database persistence of user profiles, saved network topologies, and experiment audit reports.
- Multi-stage Docker containerization and automated GitHub Actions CI/CD pipeline.
- Production hosting on AWS EC2 in region `us-east-1` with unified AWS Resource Group tracking.

4.2 Out-of-Scope Boundaries (Guarding Against Cost & Complexity Sprawl)

- **NAT Gateways:** AWS NAT Gateways incur a continuous hourly charge of approximately $0.045 per hour plus data processing fees. To protect cloud operational budgets, private subnets in the base lab model are routed locally without automated NAT Gateway provisioning.
- **IPv6 Dual-Stack Routing:** Initial platform scope focuses exclusively on IPv4 CIDR mathematics and routing.
- **Transit Gateway (TGW) & VPC Peering:** Cross-VPC and hybrid interconnect analysis is deferred to future architectural iterations.
- **Stateless Network ACLs (NACLs):** Diagnostics focus on stateful Layer 4 Security Groups, which represent the vast majority of application connectivity issues in AWS.
- **Arbitrary Shell Execution:** The Terraform runner strictly prohibits user-provided shell scripts or unverified binary execution.

---

5. Complete Technology Stack & Architectural Principles

CloudOps Lab is engineered using modern, production-grade technologies selected for high performance, strict typing, asynchronous throughput, and robust security:

```
CloudOps Lab Production Technology Stack
├── Client Presentation: React 18, Vite 5, Semantic CSS Modules,
Inter & JetBrains Mono Fonts
├── Core Backend Engine: Python 3.11, FastAPI, Pydantic v2,
Starlette, Uvicorn ASGI Server
├── Infrastructure as Code: HashiCorp Terraform CLI v1.8.5 (Sandboxed
Subprocess Runner)
├── Cloud Provider SDK: AWS Boto3 (EC2, VPC, Resource Groups, Pricing
SDKs)
├── Security & Cryptography: Argon2id (argon2-cffi), HttpOnly Session
Cookies, itsdangerous
├── Relational Persistence: SQLite 3 with Foreign Key Enforcement &
WAL (Write-Ahead Logging)
├── Outbound Email Engine: Python smtplib with TLS 1.3 over Gmail
SMTP (Port 587)
├── Containerization: Multi-Stage Docker Build (Node 20 Alpine
Builder + Python 3.9 Slim Runtime)
└── CI/CD Automation: GitHub Actions (Ubuntu Latest, Python 3.11,
Node 20, Terraform 1.8.5)
```

5.1 Technology Selection Rationale

Architectural Tier	Selected Technology	Technical Justification
Frontend Framework	React 18 (Vite SPA)	Delivers single-page application (SPA) responsiveness, component-level encapsulation for network nodes, and immediate client-side state transitions.
Backend REST Engine	FastAPI (Python 3.11)	Offers high-performance asynchronous request handling (via Starlette and AnyIO), native data validation via Pydantic v2 schemas, and automatic OpenAPI documentation.
Network Math Engine	Python `ipaddress` Standard Library	Provides mathematically rigorous bitwise IPv4 arithmetic, prefix boundary masking, and pairwise overlap collision algorithms without external library dependencies.
IaC Provisioning	HashiCorp Terraform CLI 1.8.5	Industry standard for declarative infrastructure. Bundled directly into the runtime container to ensure identical execution between local testing and AWS EC2.
Cloud Integration	AWS Boto3 SDK	The official AWS SDK for Python, providing robust API clients for querying VPCs, subnets, route tables, security groups, and resource tags.

Architectural Tier	Selected Technology	Technical Justification
Password Hashing	Argon2id (`argon2-cffi`)	Winner of the Password Hashing Competition (PHC). Provides memory-hard resistance against GPU- and ASIC-based brute-force attacks, surpassing legacy bcrypt and PBKDF2.
Relational Storage	SQLite 3	Embedded, zero-configuration ACID-compliant database engine. Configured with foreign key cascades and WAL mode, providing microsecond query latency and simple backup portability.
Continuous Integration	GitHub Actions	Automatically validates code formatting (`terraform fmt`), runs 55 unit tests (`pytest`), and verifies frontend asset compilation (`npm run build`) on every commit.

6. System Architecture & Multi-Tier Module Decomposition

The platform is designed according to a clean, four-tier decoupled architecture: Client Presentation Layer, API & Orchestration Layer, Persistence Layer, and Cloud Infrastructure Layer.

6.2 Data Flow Lifecycle

1. **Network Definition:** A user submits an IPv4 VPC configuration through `NetworkPlanner.jsx`.
2. **Mathematical Validation:** `backend/app/network.py` validates CIDR boundaries, calculates available host capacity (minus the 5 AWS-reserved IPs), and confirms that subnets do not intersect.
3. **Session Ingestion:** In `TroubleshootingWorkflow.jsx`, the network model is loaded into active session memory (`workflow_engine.py`).
4. **Diagnostic Scan:** `diagnostics.py` runs rule audits against the model or live cloud APIs, generating structured finding payloads.
5. **RCA Synthesis:** `rca_engine.py` evaluates finding dependencies, builds a multi-factor evidence chain, and calculates certainty metrics.
6. **Remediation & Guardrail:** The workflow engine generates the Terraform HCL diff. The user inspects the diff and submits the safety token `CONFIRM_REMEDIATION`.
7. **Execution & Verification:** `terraform_runner.py` applies the patch. The diagnostic suite executes a post-fix audit to confirm zero remaining failures.
8. **Compliance Commit:** `report_service.py` writes the certified post-mortem report to `experiments` in SQLite, available for Markdown and JSON export.

---

7. Pre-Deployment Network Planning & CIDR Mathematics Engine

The Network Planning Engine (`backend/app/network.py`) provides pre-deployment mathematical validation for IPv4 address spaces. In cloud architecture, allocating invalid or colliding CIDR blocks causes silent routing conflicts that require extensive downtime to rectify once workloads are live.

7.1 RFC 1918 Private IPv4 Address Space Invariants

CloudOps Lab enforces strict adherence to RFC 1918 private IPv4 allocations: * **Class A Block:** `10.0.0.0` to `10.255.255.255` (`10.0.0.0/8`) * **Class B Block:** `172.16.0.0` to `172.31.255.255` (`172.16.0.0/12`) * **Class C Block:** `192.168.0.0` to `192.168.255.255` (`192.168.0.0/16`)

The engine enforces AWS prefix boundaries: * **Minimum VPC Size:** `/28` (16 total IPv4 addresses). * **Maximum VPC Size:** `/16` (65,536 total IPv4 addresses).

7.2 The AWS 5-Reserved IP Address Mathematical Policy

In standard on-premises networking (RFC 1918), each subnet reserves exactly 2 IP addresses: the network identifier ($2^0 - 1$) and the broadcast address ($2^n - 1$).

In Amazon Web Services, exactly five IP addresses are automatically reserved in every subnet:

$$\text{Usable Capacity} = 2^{(32-P)} - 5$$

Where P represents the CIDR prefix length ($/16 \leq P \leq /28$).

```
Detailed AWS Reserved Address Mapping (Example: 10.0.1.0/24)
```

Address Offset	Technical Role	Routable	AWS Function
Network + 0	10.0.1.0 (Network ID)	No	Subnet boundary identifier
Network + 1	10.0.1.1 (VPC Router)	Yes	Default gateway for subnet
Network + 2	10.0.1.2 (DNS Resolver)	Yes	AmazonProvidedDNS server
Network + 3	10.0.1.3 (AWS Future Use)	No	Reserved by AWS for features
Network + 4	10.0.1.4 (First Usable Host)	Yes	First assignable compute ENI
...	[251 Usable Host Addresses]	Yes	Dynamic/Static instance IPs
Network + 254	10.0.1.254 (Last Usable Host)	Yes	Last assignable compute ENI
Network + 255	10.0.1.255 (Broadcast ID)	No	Subnet broadcast (Unused)

7.3 Pairwise Subnet Collision Detection Algorithm

To prevent routing ambiguity, no two subnets within a VPC may share any portion of their address spaces. The engine executes a pairwise intersection algorithm:

$$\forall i, j \in [1, N], \quad i \neq j \implies \text{Subnet}_i \cap \text{Subnet}_j = \emptyset$$

```python
# Critical Code Snippet: Pairwise Overlap Collision Detection
def check_subnet_overlaps(subnets: List[SubnetAllocation]) -> None
    for i in range len(subnets)):
        for j in range i + 1  len(subnets)):
            net1 = ipaddress.ip_network(subnets[i].cidr,
strict=False)
            net2 = ipaddress.ip_network(subnets[j].cidr,
strict=False)
            if net1.overlaps(net2):
                raise ValidationError(
                    f"Subnet '{subnets[i].name}' ({net1}) overlaps
with '{subnets[j].name}' ({net2})"
                )
```

If an operator defines `public-subnet` as `10.0.1.0/24` and attempts to allocate `10.0.1.128/25` to `private-subnet`, the engine detects that `10.0.1.128/25` is an internal subset of `10.0.1.0/24`, immediately blocking

the plan before Terraform execution.

8. The 10-Point Automated Diagnostic Audit Catalogue

When CloudOps Lab analyzes an active or proposed AWS network, it invokes the **Automated Diagnostic Suite** (`backend/app/diagnostics.py`). This engine evaluates the infrastructure against a deterministic 10-point rule catalogue covering all four networking layers:

```
[Layer 1: CIDR Mathematics]
   ├── CIDR-001: Valid RFC 1918 IPv4 CIDR Allocation
   └── CIDR-002: Host Bits Zero Invariant Check
[Layer 2: VPC Boundary & Perimeter Gateway]
   ├── VPC-001:  Subnet Containment & Overlap Verification
   └── IGW-001:  Internet Gateway Attachment State
[Layer 3: Route Tables & Subnet Tiers]
   ├── ROUTE-001: Public Tier Default Route to IGW (0.0.0.0/0)
   └── ROUTE-002: Private Tier Perimeter Isolation
[Layer 4: Security Groups & Firewalls]
   ├── SG-001:   Web Ingress HTTPS Rule Authorization (Port 443)
   └── SG-002:   Unrestricted SSH Access Audit (Port 22 from
0.0.0.0/0)
[Operational Probing & Interface Diagnostics]
   ├── LIVE-001: Outbound DNS & ICMP Reachability Probe
   └── LIVE-002: Inbound TCP Handshake Probe
```

8.1 Diagnostic Catalogue Specification Matrix

Rule ID	Layer	Rule Name	Evaluation Invariant	Failure Trigger Condition	Rer Gui
`CIDR-001`	Layer 1	Valid RFC 1918 CIDR	IPv4 address string must be valid syntax with prefix $/16 \leq P \leq /28$.	Non-standard format or prefix out of AWS bounds.	Re- net RF((`1`
`CIDR-002`	Layer 1	Host Bits Zero Check	Bitwise host portion of CIDR address must evaluate strictly to zero.	Host bits set in network mask (e.g., `10.0.1.5/24`).	Ma zer `10` to `10`
`VPC-001`	Layer 2	Subnet Containment & Collision	All subnets must reside inside parent VPC; zero subnets may intersect.	Subnet out of VPC range or colliding with adjacent subnet.	Adj CID elin
`IGW-001`	Perimeter	IGW Attachment State	Internet Gateway must exist and report state `attached` to VPC.	Gateway state is `detached` or no gateway attached.	Att Gat usi Ter
`ROUTE-001`	Layer 3	Public Route to IGW	Route table associated with public subnet must contain `0.0.0.0/0 → igw`.	Public subnet lacks `0.0.0.0/0` next-hop to IGW.	Ad rou IGW tab
`ROUTE-002`	Layer 3	Private Route Isolation	Private route table must NOT contain	Private subnet has route	Rer rou

Rule ID	Layer	Rule Name	Evaluation Invariant	Failure Trigger Condition	Rer Gui
			direct route to Internet Gateway.	pointing directly to IGW.	rou isol
`SG-001`	Layer 4	Ingress HTTPS Port 443	Web security group must authorize inbound TCP port 443.	Security group permits port 80 but omits port 443.	Ad aut for
`SG-002`	Layer 4	Unrestricted SSH Prevention	Ingress rules must not allow `0.0.0.0/0` on port 22 (SSH).	Ingress rule permits SSH from `0.0.0.0/0` to all internet.	Res acc bas VPI
`LIVE-001`	Operational	Egress Resolver Probe	Target instances must be capable of resolving outbound DNS/ICMP.	Ping or socket timeout to public resolver (`1.1.1.1`).	Ins sec rule tab
`LIVE-002`	Operational	Inbound TCP Handshake	TCP handshake to target instance port 80/443 must succeed.	Inbound SYN packet rejected or timed out at ENI.	Re- fire rule pro

```python
# Critical Code Snippet: Diagnostic Rule Evaluation Engine
def evaluate_diagnostic_rule(rule_id: str, context: NetworkLabModel)
        -> DiagnosticFinding:
    if rule_id == "SG-001":
        has_https = any(
            rule.from_port <= 443 <= rule.to_port and rule.protocol
        == "tcp"
            for sg in context.security_groups
            for rule in sg.ingress_rules
        )
        return DiagnosticFinding(
            check_id="SG-001",
            status="PASS" if has_https else "FAIL",
            severity="HIGH" if not has_https else "INFO",
            title="Web Ingress HTTPS Rule Authorization"
            evidence="Port 443 is permitted" if has_https else
        "Inbound port 443 omitted from security group"
        )
```

9. Deterministic Root-Cause Analysis (RCA) Engine

Traditional log search engines and APM monitoring tools merely report symptoms (e.g., *"Application health check failed on port 443"*). In cloud networking, identifying the true root cause requires an **evidence-based dependency elimination model**.

9.1 The Dependency-Elimination Model

CloudOps Lab implements a multi-layer deterministic root-cause analysis algorithm (`backend/app/rca_engine.py`). The algorithm models cloud networking as a strict dependency graph:

9.2 Evidence Chain Construction

Rather than providing an unverified hypothesis, the RCA engine builds an explicit, multi-factor evidence chain: * **Factor 1:** Confirms that Layer 1 CIDR allocation is mathematically sound. * **Factor 2:** Verifies that the Internet Gateway is reporting an `attached` state in AWS APIs. * **Factor 3:** Confirms that public route table association contains a valid `0.0.0.0/0 → igw` entry. * **Factor 4:** Observes that check `SG-001` failed due to missing port 443 in `cloudops-lab-web-sg`. * **Conclusion:** Because Layers 1, 2, and 3 are healthy, the network path to the instance Elastic Network Interface (ENI) is fully functional. The outage is isolated exclusively to the stateful Layer 4 firewall with **100% Certainty**.

```python
# Critical Code Snippet: Multi-Factor RCA Evidence Formulation
def compute_root_cause_analysis(findings: List[DiagnosticFinding]) →
    RcaAnalysisReport:
    failed_map = {f.check_id: f for f in findings if f.status in
        ("FAIL", "ERROR")}
    if "SG-001" in failed_map and "ROUTE-001" not in failed_map and
        "IGW-001" not in failed_map:
        return RcaAnalysisReport(
            primary_cause_id="RCA-SG-MISSING-HTTPS",
            title="Missing Inbound HTTPS (Port 443) Authorization",
            confidence_score=1.0,
            certainty_level="DEFINITIVE",
            evidence_chain=[
                "Layer 1: CIDR allocation and containment invariants
        verified",
                "Layer 2: Internet Gateway is ATTACHED to VPC
        boundary",
                "Layer 3: Default route (0.0.0.0/0 → IGW) is
        operational",
                "Layer 4: Security group web-sg permits port 80 but
        omits port 443"
            ]
        )
```

10. Declarative Infrastructure as Code (Terraform) Remediation Engine

A fundamental principle of modern Site Reliability Engineering is that **production infrastructure must never be modified by manual clicks in a cloud console**. Console changes create configuration drift, violate compliance tracking, and cannot be audited in version control.

CloudOps Lab solves this by coupling automated root-cause analysis with **declarative Terraform code generation** (`backend/app/terraform_runner.py`).

10.1 Synthesizing Declarative Remediation Diff

When an outage scenario (such as `RCA-SG-MISSING-HTTPS`) is isolated, the platform synthesizes the exact HashiCorp Terraform configuration patch:

```
# Proposed Terraform Remediation: infrastructure/security.tf
# Target Resource: aws_security_group.web_sg
+ ingress {
+   description = "Allow inbound HTTPS traffic from approved sources"
+   from_port   = 443
+   to_port     = 443
+   protocol    = "tcp"
+   cidr_blocks = ["0.0.0.0/0"]
+ }
```

10.2 Sandboxed Execution Security Architecture

Running infrastructure tools programmatically introduces security considerations. CloudOps Lab isolates Terraform execution through multi-layer security guardrails: 1. **No Shell Invocations (`shell=False`):** Commands are executed directly as process vectors (`["terraform", "apply", "-auto-approve"]`), completely eliminating shell injection vulnerabilities (`bash -c`, command concatenation, or pipes). 2. **Process Whitelisting:** The execution engine allows only four specific Terraform operations: `init`, `validate`, `plan`, and `apply`. 3. **Execution Directory Sandboxing:** Commands are strictly confined to the `infrastructure/` directory; access to parent directories or system paths is blocked. 4. **Input**

Sanitization: All dynamic parameters (e.g., CIDRs, port numbers, resource identifiers) are validated against strict alphanumeric regex patterns prior to argument assembly.

```
# Critical Code Snippet: Sandboxed Terraform Execution
def run_sandboxed_terraform(action: str, target_dir: str =
        "infrastructure") -> TerraformRunResponse:
    if action not in ALLOWED_TF_ACTIONS:
        raise SecurityException(f"Unauthorized Terraform action:
        {action}")
    cmd = ["terraform", action]
    if action == "apply":
        cmd.append("-auto-approve")
    result = subprocess.run(cmd, cwd=target_dir, capture_output=True,
        text=True, shell=False)
    return TerraformRunResponse(exit_code=result.returncode,
        stdout=result.stdout, stderr=result.stderr)
```

11. Human-in-the-Loop Safety Guardrails & Access Governance

In enterprise Site Reliability Engineering, completely autonomous, unreviewed writes to production environments represent a severe operational hazard. An automated remediation agent executing without human validation can cause catastrophic cascading failures if it acts upon false positives.

11.1 The Human-in-the-Loop Architecture

CloudOps Lab enforces a strict **Human-in-the-Loop Safety Guardrail** at Step 5 of the recovery pipeline:

11.2 Safety Token Specification

To apply changes to target cloud infrastructure: 1. The operator must inspect the green code diff, review the technical rationale, and evaluate the security implications. 2. The operator must explicitly type the required confirmation string:

$$\text{Safety Token} = \texttt{"CONFIRM_REMEDIATION"}$$

3. The backend validates the token string identically. Any variation (whitespace, incorrect casing, or empty payload) immediately rejects the request with HTTP 400.

---

12. The End-to-End 7-Step Self-Healing Recovery Workflow

The signature capability of CloudOps Lab is the **7-Step Troubleshooting & Recovery Engine** (`backend/app/workflow_engine.py`). It coordinates the entire lifecycle of cloud network fault handling:

12.1 Detailed Workflow State Transitions

Step 1: Target Network Lab Model

- **Operational Goal:** Ingests the baseline AWS architecture.
- **Component State:** VPC `10.0.0.0/16`, attached `igw-0a1b2c3d4e`, public subnet (`10.0.1.0/24`), private subnet (`10.0.2.0/24`), and security group `cloudops-lab-web-sg`.
- **State Invariant:** All 10 baseline diagnostic checks pass; the system verifies healthy operational state.

Step 2: Controlled Chaos Fault Injection

- **Operational Goal:** Injects a realistic cloud outage to test diagnostic observability.
- **Supported Fault Catalogue:**
  1. `sg_missing_https`: Inbound HTTPS port 443 omitted from web security group.
  2. `missing_default_route`: Default route `0.0.0.0/0` removed from public route table.
  3. `igw_detached`: Internet Gateway placed into detached state at VPC perimeter.
  4. `subnet_overlap`: Identical CIDR blocks allocated across multiple subnets.
  5. `invalid_cidr_boundary`: Non-zero host bits supplied in network CIDR.

Step 3: Diagnostic Inspection

- **Operational Goal:** Automatically audits the degraded infrastructure against the 10-point rule catalogue.

- **Output:** Flags check `SG-001` with status `FAIL` and severity `HIGH`, recording that port 443 is unassigned.

Step 4: Deterministic Root-Cause Analysis

- **Operational Goal:** Correlates findings to isolate the exact failure layer.
- **Output:** Produces cause code `RCA-SG-MISSING-HTTPS` with **100% confidence**, proving that Layer 1 CIDR, Layer 2 perimeter, and Layer 3 routing are fully healthy.

Step 5: Proposed Terraform Remediation

- **Operational Goal:** Synthesizes declarative Infrastructure as Code to remediate the outage.
- **Execution:** Displays HCL diff. Enforces human confirmation guardrail (`CONFIRM_REMEDIATION`). Upon validation, applies the change via sandboxed execution.

Step 6: Recovery Verification & Empirical Proof

- **Operational Goal:** Verifies that the remediation cleared the failure without introducing regressions.
- **Execution:** The diagnostic suite re-evaluates all 10 checks.
- **Certification:** Confirms `Resolved: SG-001` and `Remaining Failures: 0`, awarding **Recovery Certified (100%)**.

Step 7: Certified Experiment Audit Report

- **Operational Goal:** Archives an immutable post-mortem report for compliance.
- **Execution:** Generates session report with timestamps, user identity, before/after diffs, and verification metrics. Writes to SQLite and exports to Markdown and JSON.

---

13. Hop-by-Hop Packet Path Tracer & Cloud Resource Inventory

CloudOps Lab provides deep visibility into cloud traffic routing via the **Hop-by-Hop Packet Path Tracer** (`backend/app/network_model.py`).

13.1 Sequential Hop Inspection Logic

1. **Hop 1 (Perimeter Gateway):** Evaluates if an Internet Gateway is attached to the VPC. If detached, the trace halts with verdict `BLOCKED` .
2. **Hop 2 (Layer 3 Route Table):** Evaluates if a default route (`0.0.0.0/0`) directs ingress/egress to the gateway. If missing, the packet is flagged as dropped due to routing blackhole.
3. **Hop 3 (Layer 4 Security Group):** Evaluates stateful security group rules against protocol and port. If the port is unauthorized, the packet is flagged as blocked by security policy.
4. **Hop 4 (Target Host):** Upon passing Hops 1 through 3, the packet reaches the network interface with status `PERMITTED` .

---

14. Database Schema & Entity-Relationship Modeling (ER Diagram)

The persistence layer uses SQLite 3 configured with foreign key enforcement and Write-Ahead Logging (WAL) for high concurrency and zero-configuration portability.

15. Authentication, Cryptography & Session Security Architecture

CloudOps Lab implements modern, defense-in-depth security across authentication, authorization, session persistence, and data transmission (`backend/app/security.py`, `backend/app/auth_service.py`).

15.1 Cryptographic Password Hashing: Argon2id

In adherence to the modern cryptographic consensus and RFC 9106, the platform utilizes **Argon2id** (via `argon2-cffi`), the winner of the Password Hashing Competition. Argon2id combines resistance against side-channel cache attacks (from Argon2i) and high-throughput GPU/ASIC cracking arrays (from Argon2d).

- **Algorithm:** Argon2id
- **Memory Cost (m):** 65,536 KiB (64 MiB)
- **Time Cost (t):** 3 iterations
- **Parallelism (p):** 4 parallel execution lanes
- **Salt:** 16 bytes of cryptographically secure pseudo-random entropy (`os.urandom`)

Passwords are never logged or stored in plaintext. Password policies enforce length boundaries ($12 \leq \text{length} \leq 128$ characters) without arbitrary character constraints that encourage weak patterns.

15.2 Server-Managed Session Security

Unlike stateless JWTs (which cannot be revoked server-side without complex blacklisting), CloudOps Lab utilizes **secure server-managed sessions**: * **Session ID Generation:** 64 hexadecimal characters generated via `secrets.token_hex(32)` providing 256 bits of entropy. * **Cookie Delivery Parameters:** * `HttpOnly`: Prevents client-side JavaScript access, mitigating Cross-Site Scripting (XSS) session theft. * `SameSite=Lax`: Defends against Cross-Site Request Forgery (CSRF) across external origins. * `Secure`: Enforced in production to guarantee transmission over encrypted HTTPS channels. * **Session Rotation:** Session identifiers are

rotated upon successful authentication to eliminate session fixation vulnerabilities. * **Session Lifetime:** Enforces an absolute 12-hour session boundary with a 30-minute idle expiration timer.

15.3 Outbound Email Delivery & Account Verification

- **Engine:** Python standard `smtplib` executing asynchronous transport over Gmail SMTP (`smtp.gmail.com:587`) using TLS 1.3 encryption.
- **Token Policy:** Verification and password-reset links utilize single-use tokens expiring in 30 minutes, stored hashed in SQLite (`auth_tokens`).

```python
# Critical Code Snippet: Argon2id Cryptographic Verification
from argon2 import PasswordHasher
ph = PasswordHasher(time_cost=3, memory_cost=65536, parallelism=4)

def hash_password(plaintext: str) -> str:
    return ph.hash(plaintext)

def verify_password(plaintext: str, password_hash: str) -> bool
    try:
        return ph.verify(password_hash, plaintext)
    except Exception
        return False
```

16. Production Containerization & Multi-Stage Docker Architecture

To eliminate environment divergence between local environments and cloud hosts, CloudOps Lab is containerized via a production **multi-stage Dockerfile** (`Dockerfile`).

16.1 Multi-Architecture Dynamic Binary Resolution

The production Dockerfile automatically queries the underlying CPU architecture during image build:

```
# Dynamic multi-architecture Terraform binary installation
RUN ARCH=$(dpkg --print-architecture) \
    && curl -fsSL \
        "https://releases.hashicorp.com/terraform/1.8.5/terraform_1.8.5_linux_${ARCH}.zip" \
        -o /tmp/terraform.zip \
    && unzip /tmp/terraform.zip -d /usr/local/bin \
    && rm /tmp/terraform.zip \
    && terraform version
```

This enables seamless, native compilation on both **ARM64** (Apple Silicon M-series processors) and **AMD64/x86_64** (AWS EC2 production compute instances).

16.2 Automated Docker Health Checks

The container includes automated health probing:

```
HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 \
  CMD curl -f http://localhost:8000/health || exit 1
```

Docker polls the `/health` endpoint every 30 seconds. If three consecutive checks fail, Docker marks the container `unhealthy` and notifies the host daemon.

---

17. Automated Testing Suite & Continuous Integration Pipeline

To ensure reliability across all platform modules, CloudOps Lab maintains a **continuous integration pipeline** powered by GitHub Actions (`.github/workflows/checks.yml`).

17.1 Test Suite Breakdown (55 Automated Unit & Integration Tests)

The backend test suite (`backend/tests/`) covers all critical platform components: 1. `test_network.py`: Tests bitmask boundaries, available host counts (251 usable hosts per $/24$), and out-of-bounds prefix rejections. 2. `test_overlap.py`: Validates pairwise subnet overlap collision detection. 3. `test_diagnostics.py`: Evaluates all 10 diagnostic rules under baseline, degraded, and remediated states. 4. `test_rca_engine.py`: Asserts deterministic layer elimination, verifying that Layer 3 routing health isolates Layer 4 firewall omissions with 100% confidence. 5. `test_terraform_runner.py`: Tests sandboxed execution security, action allowlists, and execution output parsing. 6. `test_auth_and_persisted_workflow.py`: Validates Argon2id hashing, user creation, session rotation, and experiment database persistence.

---

18. Live Cloud Infrastructure & AWS Resource Group Organization

CloudOps Lab is deployed live in Amazon Web Services in Region `us-east-1` **(US East - N. Virginia)**.

18.1 Host Compute Instance Specification

- **Instance Identifier:** `i-05deff2eb16bab7d4`
- **Instance Type:** `t3.micro` (2 vCPU, 1 GiB RAM)
- **Operating System:** Ubuntu Linux 22.04 LTS (x86_64)
- **Public IPv4 Address:** `44.199.232.34`
- **Public DNS:** `ec2-44-199-232-34.compute-1.amazonaws.com`
- **Network Interface:** `eni-023ba6ab9c3af7de0`
- **Persistent Storage:** 8 GiB GP3 Elastic Block Store (`vol-0b8f52992f675aef6`)

18.2 AWS Resource Group Governance (`cloudops-lab-resources`)

All cloud assets provisioned for CloudOps Lab are organized under a centralized **AWS Resource Group** using tag-based query filtering (`Project: CloudOpsLab`):

```
AWS Resource Group: cloudops-lab-resources (Region: us-east-1)
├── AWS::EC2::Instance         → i-05deff2eb16bab7d4 (cloudops-lab-production)
├── AWS::EC2::SecurityGroup    → sg-0d0b43a2b00e18ad1 (cloudops-lab-security-group)
├── AWS::EC2::NetworkInterface → eni-023ba6ab9c3af7de0 (cloudops-lab-network-interface)
└── AWS::EC2::Volume           → vol-0b8f52992f675aef6 (cloudops-lab-ebs-storage)
```

This resource grouping allows infrastructure teams to monitor compute health, storage metrics, firewall rules, and billing attribution from a single unified AWS console view:
AWS Console URL: `https://us-east-1.console.aws.amazon.com/resource-groups/group/cloudops-lab-resources?region=us-east-1`

---

19. Audit Reporting, Compliance & Regulatory Verification

In enterprise cloud governance (SOC 2 Type II, ISO 27001, and the AWS Well-Architected Framework), all infrastructure alterations must produce an immutable audit trail.

19.1 Certified Audit Report Structure

Upon completion of Step 7 in the Recovery Engine, CloudOps Lab generates an immutable audit record: * **Unique Session Identifier:** Cryptographically random session ID (e.g., `wf-session-1793646114`). * **Timestamp & Attribution:** Exact UTC execution timestamp and authenticated engineer identity (`ashuchaudhary112004@gmail.com`). * **Degraded State Findings:** Structured list of all failed diagnostic checks, severity levels, and raw failure evidence. * **Root-Cause Analysis Proof:** Documented RCA cause code, confidence percentage, and multi-factor evidence chain. * **Remediation Code Diff:** Exact Terraform HCL code applied to infrastructure. * **Safety Confirmation Verification:** Verification that the human confirmation guardrail token was supplied. * **Empirical Verification Proof:** Certified metric proving 0 remaining failures post-apply.

19.2 Export Formats

Reports are immediately available for export via REST API: * **Markdown (`.md`):** Formatted in GitHub Flavored Markdown for direct inclusion in post-incident documentation, Jira tickets, and GitHub pull requests (`GET /api/reports/{id}/export?format=md`). * **JSON (`.json`):** Machine-readable structured payload for ingestion into SIEM tools, Splunk, Datadog, or external compliance engines (`GET /api/reports/{id}/export?format=json`).

---

20. Operational Runbook & Administrator Guide

20.1 Localhost Startup & Execution Runbook

To launch CloudOps Lab locally on a developer or analyst machine:

```
# 1. Clone the production repository
git clone https://github.com/a7hu-15/cloud_ops.git
cd cloud-ops

# 2. Initialize Python virtual environment & dependencies
python3 -m venv .venv
source .venv/bin/activate
pip install -r backend/requirements.txt

# 3. Build optimized frontend production bundle
cd frontend && npm install && npm run build && cd ..

# 4. Launch unified backend & frontend server on port 8080
.venv/bin/uvicorn backend.app.main:app --host 127.0.0.1 --port 8080
```

Open `http://localhost:8080` in your browser.

20.2 Stopping the Local Server

- To stop the active server in terminal: Press `Ctrl + C`.
- To stop background instances running on port 8080:

```
kill -9 $(lsof -t -i :8080)
```

20.3 Executing the Automated Test Suite

To verify all 55 unit and integration tests locally:

```
PYTHONPATH=. pytest backend/tests/ -v
```

20.4 Docker Container Execution

To launch via containerized Docker daemon:

```
docker run -d --name cloudops-lab -p 8080:8080 cloudops-lab:latest
```

21. References, Industry Standards & Citations

1. **Internet Engineering Task Force (IETF):**
   - *RFC 1918:* Address Allocation for Private Internets (Y. Rekhter, B. Moskowitz, D. Karrenberg, G. J. de Groot, E. Lear, February 1996).
   - *RFC 791:* Internet Protocol - DARPA Internet Program Protocol Specification (J. Postel, September 1981).
   - *RFC 9106:* Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications (A. Biryukov, D. Dinu, D. Khovratovich, S. Josefsson, February 2021).
   - *RFC 4632:* Classless Inter-domain Routing (CIDR): The Internet Address Assignment and Aggregation Plan (V. Fuller, T. Li, August 2006).
2. **Amazon Web Services (AWS) Architectural Documentation:**
   - *AWS Well-Architected Framework:* Reliability Pillar (Amazon Web Services, Inc., 2024).
   - *AWS Well-Architected Framework:* Security Pillar (Amazon Web Services, Inc., 2024).
   - *Amazon Virtual Private Cloud User Guide:* VPCs and Subnets Subnet Sizing and Reserved IP Addresses (AWS Documentation, 2026).
   - *Amazon EC2 Security Groups User Guide:* Security Group Ingress and Egress Rule Protocols (AWS Documentation, 2026).
   - *Boto3 Documentation:* Amazon Web Services SDK for Python (v1.34+).
3. **Infrastructure as Code & Site Reliability Engineering Standards:**
   - *HashiCorp Terraform Documentation:* Terraform Language Specification, State Management, and CLI Execution Internals (HashiCorp, 2024).
   - *Site Reliability Engineering:* How Google Runs Production Systems (B. Beyer, C. Jones, J. Petoff, N. R. Murphy, O'Reilly Media, 2016).
   - *The Site Reliability Workbook:* Practical Ways to Implement SRE (B. Beyer, N. R. Murphy, D. K. Rensin, K. Kawahara, S. Thorne, O'Reilly Media, 2018).
4. **National Institute of Standards and Technology (NIST):**
   - *NIST Special Publication 800-145:* The NIST Definition of Cloud Computing (P. Mell, T. Grance, September 2011).

- *NIST Special Publication 800-53 (Rev. 5):* Security and Privacy Controls for Information Systems and Organizations (Access Control & Incident Response Families).

5. **Open Source Core Specifications:**
   - *FastAPI:* High-Performance Web Framework for Python (Sebastián Ramírez, 2018–2026).
   - *React:* A JavaScript Library for Building User Interfaces (Meta Open Source, 2013–2026).
   - *SQLite:* An In-Process Relational Database Engine (D. Richard Hipp, 2000–2026).

End of Comprehensive Technical Report — CloudOps Lab Systems Architecture Specification